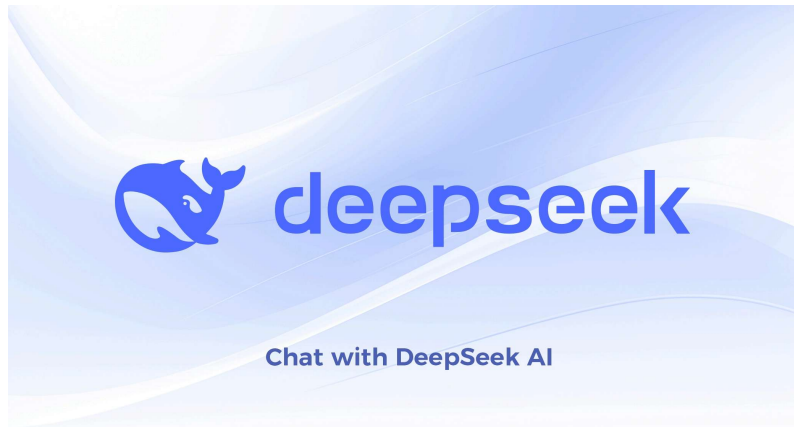


DOM Concept Explained Simply - DeepSeek



Complete Guide to 4 Essential DOM Events

Here's a **complete, runnable explanation** for each event with examples you can copy-paste and try.

1. MOUSEOUT EVENT

What it does: Fires when mouse cursor **leaves** an element (including its children).

Difference between mouseleave and mouseout:

- **mouseout**: Triggers when leaving element OR any child element (bubbles)
- **mouseleave**: Triggers only when leaving the element itself (doesn't bubble)

Complete Example:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Mouseout Event Demo</title>
  <style>
    .parent {
      width: 400px;
      height: 300px;
      background-color: lightblue;
      padding: 20px;
      margin: 20px;
      border: 2px solid black;
    }
    .child {
      width: 150px;
      height: 100px;
      background-color: coral;
      margin: 20px;
      padding: 10px;
      border: 1px solid black;
    }
    .info {
      margin-top: 20px;
      padding: 10px;
      background-color: #f0f0f0;
      font-family: monospace;
    }
    .highlight {
      background-color: yellow;
    }
  </style>
</head>
<body>
  <h2>Mouseout Event Demo</h2>

  <div class="parent" id="parent">
    <h3>Parent Element</h3>
    <div class="child" id="child">
      Child Element
    </div>
    <p>Hover and move mouse out to see event</p>
  </div>

  <div class="info" id="info">
    <strong>Events Log:</strong><br>
    <span id="log"></span>
  </div>

  <script>
    let parent = document.getElementById("parent");
```

```

let child = document.getElementById("child");
let logSpan = document.getElementById("log");

function log(message) {
    logSpan.innerHTML += "<br>" + new
Date().toLocaleTimeString() + ": " + message;
}

parent.addEventListener("mouseout", function(e) {
    log(`MOUSEOUT: Left ${e.target.tagName} (target),
moved to ${e.relatedTarget?.tagName || 'outside'}`);
    this.style.backgroundColor = "#ffcccc";
    setTimeout(() => this.style.backgroundColor =
"lightblue", 500);
});

parent.addEventListener("mouseenter", function() {
    log("MOUSEENTER: Entered parent (mouseleave not
triggered yet)");
});

parent.addEventListener("mouseleave", function() {
    log("MOUSELEAVE: Left parent completely (does NOT
trigger on child exit)");
});

child.addEventListener("mouseout", function(e) {
    log(`🎯 CHILD MOUSEOUT: Left child, going to
${e.relatedTarget?.tagName || 'outside'}`);
    this.style.transform = "scale(0.95)";
    setTimeout(() => this.style.transform =
"scale(1)", 300);
});

setTimeout(() => {
    log("👉 Try moving mouse: Enter parent → move to
child → leave child → leave parent");
}, 1000);
</script>
</body>
</html>

```

Key Points:

- **mouseout** triggers when leaving the element AND its children

- Use `e.relatedTarget` to see where mouse is going
- Good for: dropdown menus, tooltips, hover effects cleanup

2. KEYPRESS EVENT (DEPRECATED - but good to know)

Note: `keypress` is **deprecated**. Use `keydown` instead. But let me explain both.

Difference:

- `keypress`: Only triggers for **character keys** (a, b, 1, 2, \$, etc.)
- `keydown`: Triggers for **ALL keys** (including Shift, Ctrl, Arrow keys)

Complete Example:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Keypress Event Demo</title>
  <style>
    .demo-area {
      margin: 20px;
      padding: 20px;
      border: 2px solid #333;
      border-radius: 10px;
      background-color: #f9f9f9;
    }
    input {
      width: 300px;
      padding: 10px;
      font-size: 16px;
      margin: 10px;
    }
    .output {
      background-color: #e0e0e0;
      padding: 15px;
      margin: 10px;
      font-family: monospace;
      white-space: pre-wrap;
    }
    .key-box {
      display: inline-block;
      background-color: #333;
      color: white;
      padding: 5px 10px;
      margin: 5px;
      border-radius: 5px;
      font-family: monospace;
    }
  </style>
</head>
<body>
  <h2>Keypress vs Keydown Demo</h2>

  <div class="demo-area">
    <h3>Try typing in this field:</h3>
    <input type="text" id="testInput" placeholder="Type
something here...">

    <div class="output">
      <strong>Keypress Events (character keys only):
</strong>

      <div id="keypressLog"></div>

      <strong>Keydown Events (all keys):</strong>
```

```

        <div id="keydownLog"></div>

        <strong>Keyup Events (key released):</strong>
        <div id="keyupLog"></div>
    </div>

    <div id="visualizer" style="font-size: 24px; text-align: center; padding: 20px;">
        Press any key...
    </div>
</div>

<script>
    let input = document.getElementById("testInput");
    let keypressLog =
document.getElementById("keypressLog");
    let keydownLog =
document.getElementById("keydownLog");
    let keyupLog = document.getElementById("keyupLog");
    let visualizer =
document.getElementById("visualizer");

    function addLog(element, message, color) {
        let span = document.createElement("span");
        span.textContent = message;
        span.style.display = "inline-block";
        span.style.backgroundColor = color || "#eee";
        span.style.padding = "3px 8px";
        span.style.margin = "3px";
        span.style.borderRadius = "3px";
        span.style.fontSize = "12px";
        element.appendChild(span);
        element.scrollTop = element.scrollHeight;

        setTimeout(() => {
            if(element.children.length > 15) {
                element.removeChild(element.children[0]);
            }
        }, 100);
    }

    input.addEventListener("keypress", function(e) {
        let message = `Key: ${e.key} | Code: ${e.charCode}
| Type: ${typeof e.key}`;
        addLog(keypressLog, message, "#d4edda");
        visualizer.innerHTML = `
abc
        Keypress: "${e.key}"
(charCode: ${e.charCode})`;
        visualizer.style.backgroundColor = "#d4edda";
    });

```

```

        console.log("Keypress:", {
            key: e.key,
            code: e.code,
            charCode: e.charCode,
            keyCode: e.keyCode
        });
    });

    input.addEventListener("keydown", function(e) {
        let message = `Key: ${e.key} | Code: ${e.code} |
Ctrl: ${e.ctrlKey} | Alt: ${e.altKey} | Shift: ${e.shiftKey}`;
        addLog(keydownLog, message, "#fff3cd");
        visualizer.innerHTML = `⬇️ Keydown: "${e.key}"
(code: ${e.code})`;
        visualizer.style.backgroundColor = "#fff3cd";

        if(e.key === "Enter") {
            addLog(keydownLog, "⚠️ Pressed Enter - you
could submit a form!", "#f8d7da");
        }

        if(e.key === " " || e.key === "Space") {
            addLog(keydownLog, "⚠️ Space pressed - could
trigger button!", "#f8d7da");
        }
    });

    input.addEventListener("keyup", function(e) {
        let message = `Released: ${e.key}`;
        addLog(keyupLog, message, "#cce5ff");
        visualizer.innerHTML = `⬆️ Keyup: "${e.key}"
released`;
        visualizer.style.backgroundColor = "#cce5ff";

        setTimeout(() => {
            visualizer.innerHTML = "Press any key...";
            visualizer.style.backgroundColor = "";
        }, 1000);
    });

    document.addEventListener("keydown", function(e) {
        if(e.ctrlKey && e.key === 's') {
            e.preventDefault();
            alert("Ctrl+S prevented! Use this for save
shortcuts.");
        }
    });

```

```

        if(e.altKey && e.key === 'a') {
            e.preventDefault();
            alert("Alt+A pressed! Custom shortcut
triggered.");
        }
    });

    let note = document.createElement("div");
    note.style.backgroundColor = "#ffeb3b";
    note.style.padding = "10px";
    note.style.margin = "10px";
    note.style.borderRadius = "5px";
    note.innerHTML = "⚠️ <strong>Note:</strong> 'keypress'
event is deprecated. Use 'keydown' for modern apps!";
    document.querySelector(".demo-
area").insertBefore(note, document.querySelector(".output"));

    console.log("Key Events Reference:");
    console.log("- keypress: Only character keys (a, b, 1,
2, $, etc.)");
    console.log("- keydown: All keys, including
modifiers");
    console.log("- keyup: When key is released");
</script>
</body>
</html>

```

Key Takeaways:

- ❌ **keypress** is **deprecated** - don't use in production
- ✅ Use **keydown** for most cases
- Check **e.key**, **e.code**, **e.ctrlKey**, etc.
- Use **e.preventDefault()** to block keys

3. SCROLL EVENT

What it does: Fires when user scrolls an element or the page.

Performance note: Scroll events fire MANY times - use throttling!

Complete Example:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Scroll Event Demo</title>
  <style>
    body {
      margin: 0;
      font-family: Arial;
    }
    .navbar {
      position: fixed;
      top: 0;
      width: 100%;
      background-color: #333;
      color: white;
      padding: 15px;
      text-align: center;
      transition: all 0.3s;
      z-index: 100;
    }
    .navbar.scrolled {
      background-color: #ff5722;
      padding: 8px;
      font-size: 14px;
    }
    .content {
      margin-top: 70px;
      padding: 20px;
    }
    .section {
      height: 400px;
      margin: 20px 0;
      padding: 20px;
      border-radius: 10px;
      transition: all 0.3s;
    }
    .section1 { background: linear-gradient(135deg,
#667eea 0%, #764ba2 100%); }
    .section2 { background: linear-gradient(135deg,
#f093fb 0%, #f5576c 100%); }
    .section3 { background: linear-gradient(135deg,
#4facfe 0%, #00f2fe 100%); }
    .section4 { background: linear-gradient(135deg,
#43e97b 0%, #38f9d7 100%); }

    .progress-bar {
      position: fixed;
      top: 0;
      left: 0;
      width: 0%;
```

```
        height: 4px;
        background-color: #ff5722;
        z-index: 1000;
        transition: width 0.1s;
    }

    .scroll-info {
        position: fixed;
        bottom: 20px;
        right: 20px;
        background-color: rgba(0,0,0,0.8);
        color: white;
        padding: 10px;
        border-radius: 5px;
        font-family: monospace;
        font-size: 12px;
        z-index: 100;
    }

    .back-to-top {
        position: fixed;
        bottom: 20px;
        left: 20px;
        background-color: #ff5722;
        color: white;
        border: none;
        padding: 10px 15px;
        border-radius: 5px;
        cursor: pointer;
        display: none;
        z-index: 100;
    }

    .card {
        background-color: white;
        padding: 20px;
        margin: 20px 0;
        border-radius: 8px;
        box-shadow: 0 2px 10px rgba(0,0,0,0.1);
        transform: translateX(-50px);
        opacity: 0;
        transition: all 0.5s;
    }

    .card.visible {
        transform: translateX(0);
        opacity: 1;
    }
}

</style>
</head>
<body>
```

```
<div class="progress-bar" id="progressBar"></div>

<div class="navbar" id="navbar">
  Scroll Event Demo - Watch the navbar change!
</div>

<div class="content">
  <div class="section section1" id="section1">
    <h2>Section 1</h2>
    <p>Scroll down to see different events
triggered...</p>
  </div>

  <div class="section section2" id="section2">
    <h2>Section 2</h2>
    <p>Watch the scroll position and progress bar</p>
  </div>

  <div class="section section3" id="section3">
    <h2>Section 3</h2>
    <div class="card" id="card1">
      This card animates when scrolled into view!
    </div>
    <div class="card" id="card2">
      Scroll events can trigger animations
    </div>
    <div class="card" id="card3">
      Great for lazy loading and infinite scroll
    </div>
  </div>

  <div class="section section4" id="section4">
    <h2>Section 4 - End of Demo</h2>
    <p>Scroll events are powerful but need performance
optimization!</p>
  </div>
</div>

<div class="scroll-info" id="scrollInfo">
  Scroll position: 0px<br>
  Scroll percentage: 0%
</div>

<button class="back-to-top" id="backToTop">↑ Back to
Top</button>

<script>
  let navbar = document.getElementById("navbar");
  let progressBar =
document.getElementById("progressBar");
  let scrollInfo =
```

```
document.getElementById("scrollInfo");
let backToTop = document.getElementById("backToTop");
let cards = document.querySelectorAll(".card");

function throttle(func, delay) {
  let lastCall = 0;
  return function(...args) {
    let now = new Date().getTime();
    if(now - lastCall < delay) return;
    lastCall = now;
    return func(...args);
  };
}

function debounce(func, delay) {
  let timeout;
  return function(...args) {
    clearTimeout(timeout);
    timeout = setTimeout(() => func.apply(this,
args), delay);
  };
}

function handleScroll() {
  let scrollY = window.scrollY;
  let windowHeight = window.innerHeight;
  let documentHeight =
document.documentElement.scrollHeight;
  let maxScroll = documentHeight - windowHeight;
  let scrollPercent = (scrollY / maxScroll) * 100;

  progressBar.style.width = scrollPercent + "%";

  scrollInfo.innerHTML = `
    Scroll position: ${Math.floor(scrollY)}px<br>
    Scroll percentage:
    ${Math.floor(scrollPercent)}%<br>
    Window height: ${windowHeight}px<br>
    Document height: ${documentHeight}px
  `;

  if(scrollY > 50) {
    navbar.classList.add("scrolled");
  } else {
    navbar.classList.remove("scrolled");
  }
}
```

```

    }

    if(scrollY > 300) {
        backToTop.style.display = "block";
    } else {
        backToTop.style.display = "none";
    }

    cards.forEach(card => {
        let rect = card.getBoundingClientRect();
        if(rect.top < windowHeight - 100) {
            card.classList.add("visible");
        }
    });

    let sections =
document.querySelectorAll(".section");
    sections.forEach(section => {
        let rect = section.getBoundingClientRect();
        if(rect.top < 150 && rect.bottom > 150) {
            section.style.transform = "scale(1.02)";
            section.style.boxShadow = "0 10px 30px
rgba(0,0,0,0.3)";
        } else {
            section.style.transform = "scale(1)";
            section.style.boxShadow = "none";
        }
    });

}

window.addEventListener("scroll",
throttle(handleScroll, 50));

window.addEventListener("scroll", debounce(() => {
    console.log("🌈 Scroll stopped at:",
window.scrollY);
}, 200));

backToTop.addEventListener("click", () => {
    window.scrollTo({
        top: 0,
        behavior: "smooth"
    });
});

```

```

    });
  });

  console.log("Scroll event demo ready!");
  console.log("Tips:");
  console.log("1. Watch progress bar at top");
  console.log("2. Navbar changes color after 50px");
  console.log("3. Cards animate when visible");
  console.log("4. Back to top button appears after
300px");

  let observer = new IntersectionObserver((entries) => {
    entries.forEach(entry => {
      if(entry.isIntersecting) {
        console.log(`👁️ ${entry.target.id} is
visible in viewport`);
        entry.target.style.opacity = "1";
      }
    });
  }, { threshold: 0.5 });

  document.querySelectorAll(".section").forEach(section
=> {
    observer.observe(section);
  });
</script>
</body>
</html>

```

Scroll Event Best Practices:

-  Use **throttle** (limit calls to 60fps)
-  Use **debounce** for scroll-stop actions
-  Consider **Intersection Observer** for visibility detection
-  Don't do heavy DOM operations in scroll

4. LOAD EVENT

What it does: Fires when the entire page (including images, stylesheets, scripts) has finished loading.

Difference:

- **DOMContentLoaded:** Fires when DOM is ready (faster)

- **load**: Fires when all resources are loaded (slower)

Complete Example:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Load Event Demo</title>
  <style>
    body {
      font-family: Arial;
      padding: 20px;
    }
    .loader {
      position: fixed;
      top: 0;
      left: 0;
      width: 100%;
      height: 100%;
      background: rgba(0,0,0,0.9);
      display: flex;
      justify-content: center;
      align-items: center;
      flex-direction: column;
      z-index: 9999;
      color: white;
      transition: opacity 1s;
    }
    .spinner {
      width: 50px;
      height: 50px;
      border: 5px solid #f3f3f3;
      border-top: 5px solid #ff5722;
      border-radius: 50%;
      animation: spin 1s linear infinite;
    }
    @keyframes spin {
      0% { transform: rotate(0deg); }
      100% { transform: rotate(360deg); }
    }
    .info-box {
      background-color: #f0f0f0;
      padding: 15px;
      margin: 10px 0;
      border-left: 4px solid #ff5722;
      font-family: monospace;
    }
    img {
      max-width: 200px;
      margin: 10px;
      transition: all 0.3s;
    }
    .loaded {
      border: 2px solid green;
    }
```

```

    }
    .timing {
        position: fixed;
        bottom: 10px;
        right: 10px;
        background: #333;
        color: white;
        padding: 10px;
        border-radius: 5px;
        font-size: 12px;
        font-family: monospace;
    }
</style>
</head>
<body>
    <div class="loader" id="loader">
        <div class="spinner"></div>
        <p>Loading page and resources...</p>
        <p id="loaderStatus"></p>
    </div>

    <h1>Load Event Demo</h1>

    <div class="info-box">
        <strong>🕒 Page Load Timeline:</strong>
        <div id="timeline"></div>
    </div>

    <h2>Images Loading:</h2>
    <div id="imageContainer">
        
        
        
    </div>

    <div class="info-box" id="performanceInfo"></div>

    <div class="timing" id="timing">
        Waiting...
    </div>

    <script>

        let startTime = performance.now();
        let domReadyTime = null;
        let loadTime = null;

```

```

        document.addEventListener("DOMContentLoaded",
function() {
    domReadyTime = performance.now() - startTime;
    document.getElementById("timeline").innerHTML += `
         DOMContentLoaded:
    ${domReadyTime.toFixed(2)}ms<br>
        DOM is ready but images may still be loading
    `;
    document.getElementById("loaderStatus").innerHTML
= "DOM Ready! Waiting for images...";
    console.log("DOMContentLoaded - DOM is ready!");

    document.body.style.backgroundColor = "#fafafa";
});

window.addEventListener("load", function() {
    loadTime = performance.now() - startTime;
    let loader = document.getElementById("loader");
    let timingDiv = document.getElementById("timing");

    loader.style.opacity = "0";
    setTimeout(() => {
        loader.style.display = "none";
    }, 1000);

    document.getElementById("timeline").innerHTML += `
         Load Event: ${loadTime.toFixed(2)}ms<br>
        🎉 All resources (images, CSS, etc.) are
loaded!
    `;

    timingDiv.innerHTML = `
        Total load time: ${loadTime.toFixed(2)}ms<br>
        DOM ready: ${domReadyTime.toFixed(2)}ms
    `;

    let images = document.querySelectorAll("img");
    images.forEach(img => {
        img.classList.add("loaded");
    });

    if(window.performance &&
window.performance.timing) {
        let timing = window.performance.timing;
        let loadEventEnd = timing.loadEventEnd -
timing.navigationStart;

```

```

        let domInteractive = timing.domInteractive -
timing.navigationStart;

document.getElementById("performanceInfo").innerHTML = `
        <strong>📊 Detailed Performance Metrics:
</strong><br>
        Navigation start to load event:
${loadEventEnd}ms<br>
        DOM Interactive: ${domInteractive}ms<br>
        Page loaded completely:
${loadTime.toFixed(2)}ms
        `;
    }

    console.log("LOAD - All resources loaded!");
    console.log(`Total load time:
${loadTime.toFixed(2)}ms`);
    });

    let loadedImages = 0;
    let totalImages =
document.querySelectorAll("img").length;

    document.querySelectorAll("img").forEach(img => {
        img.addEventListener("load", function() {
            loadedImages++;

document.getElementById("loaderStatus").innerHTML =
            `Loaded ${loadedImages}/${totalImages}
images...`;

            console.log(`Image loaded: ${this.src}
(${loadedImages}/${totalImages})`);
            });

            img.addEventListener("error", function() {
                console.log(`Failed to load: ${this.src}`);
                this.style.border = "2px solid red";
            });
        });

        window.addEventListener("beforeunload", function(e) {

            console.log("beforeunload - User is leaving the
page");
        });
    });

```

```

window.addEventListener("unload", function() {
    console.log("unload - Page is being closed");

});

window.addEventListener("pageshow", function(e) {
    if(e.persisted) {
        console.log("Page loaded from bfcache
(back/forward button)");
        document.getElementById("timeline").innerHTML
+= `


🔄 Page restored from cache (back/forward
button)<br>


`;
    }
});

console.log("Initial script execution");
console.log("Note: Script runs BEFORE DOMContentLoaded
and LOAD");

console.log("Current document.readyState:",
document.readyState);

document.addEventListener("readystatechange", () => {
    console.log("readyState changed to:",
document.readyState);
});
</script>

<link rel="stylesheet"
href="https://fonts.googleapis.com/css2?
family=Roboto:wght@300&display=swap">
</body>
</html>

```

Load Event Timeline:

text

1. Initial script execution (inline scripts)
2. DOMContentLoaded (DOM ready)
3. Images, stylesheets, fonts load
4. Load event (everything ready)

Use Cases:

- **DOMContentLoaded**: Initialize app, attach event listeners
- **load**: Measure performance, hide loaders, heavy initializations
- **beforeunload**: Warn users about unsaved changes
- **pageshow**: Handle back/forward navigation

COMPARISON TABLE

COMPLETE TESTING ENVIRONMENT

Save this as **events_demo.html** and open in browser:

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Complete Events Demo</title>
  <style>
    body { font-family: Arial; padding: 20px; }
    button { margin: 5px; padding: 10px; }
    .box {
      width: 200px;
      height: 200px;
      background: lightblue;
      margin: 20px;
      display: flex;
      align-items: center;
      justify-content: center;
    }
    .log {
      background: #f0f0f0;
      padding: 10px;
      height: 200px;
      overflow-y: scroll;
      font-family: monospace;
      font-size: 12px;
    }
  </style>
</head>
<body>
  <h1>Test All Events</h1>

  <div class="box" id="testBox">
    Hover me (mouseout)
  </div>

  <input type="text" id="testInput" placeholder="Type here
(keypress)">

  <div id="scrollArea" style="height: 200px; overflow-y:
scroll; border: 1px solid #ccc;">
    <div style="height: 500px;">Scroll this area</div>
  </div>

  <button id="testLoad">Test Load (reload page)</button>

  <div class="log" id="log"></div>

  <script>
    let logDiv = document.getElementById("log");

    function addLog(eventName, details) {
      let p = document.createElement("div");
```

```

        p.textContent = `${new
Date().toLocaleTimeString()} - ${eventName}: ${details}`;
        logDiv.appendChild(p);
        logDiv.scrollTop = logDiv.scrollHeight;
        console.log(eventName, details);
    }

    document.getElementById("testBox").addEventListener("mouseout",
(e) => {
        addLog("MOUSEOUT", "Mouse left the box");
    });

    document.getElementById("testInput").addEventListener("keypress",
(e) => {
        addLog("KEYPRESS", `Pressed "${e.key}" (charCode:
${e.charCode})`);
    });

    document.getElementById("scrollArea").addEventListener("scroll",
(e) => {
        addLog("SCROLL", `Scroll position:
${e.target.scrollTop}px`);
    });

    window.addEventListener("load", () => {
        addLog("LOAD", "Page and all resources loaded");
    });

    document.addEventListener("DOMContentLoaded", () => {
        addLog("DOMContentLoaded", "DOM ready, images may
still load");
    });

    addLog("INFO", "Demo ready! Test each event above.");
</script>
</body>
</html>

```

Try these exercises:

1. Modify the mouseout demo to change background color on exit
2. Create a character counter using keypress

3. Build an infinite scroll using scroll event

4. Create a loading spinner that hides on load

Each event has specific use cases - practice all of them! 🚀

Qs2. Create a button on the page using JavaScript. Add an event listener to the button that changes the button's color to green when it is clicked. Qs3. Create an input element on the page with a placeholder "enter your name" and an H2 heading on the page inside HTML. The purpose of this input element is to enter a user's name so it should only input letters from a-z, A-Z and space (all other characters should not be detected). Whenever the user inputs their name, their input should be dynamically visible inside the heading. [Please note that no other character apart from the allowed characters should be visible in the heading]

Solution to Both Questions

Here are the complete solutions with detailed explanations.

QUESTION 2: Button Color Change on Click

Requirements:

- Create button using JavaScript (not HTML)
- Add event listener
- Change button color to green when clicked

Solution 1: Basic Version

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Question 2 - Button Color Change</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      padding: 50px;
    }
    button {
      padding: 15px 30px;
      font-size: 18px;
      border: none;
      border-radius: 5px;
      cursor: pointer;
      transition: all 0.3s;
    }
  </style>
</head>
<body>
  <h1>Question 2 Solution</h1>
  <div id="button-container"></div>

  <script>

    let button = document.createElement("button");
    button.textContent = "Click Me!";

    button.addEventListener("click", function() {
      button.style.backgroundColor = "green";
      button.style.color = "white";
    });

    document.getElementById("button-
container").appendChild(button);
  </script>
</body>
</html>
```

Solution 2: Enhanced Version (with toggle effect)

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Question 2 - Enhanced Button</title>
  <style>
    body {
      font-family: Arial;
      text-align: center;
      padding: 50px;
      background: #f0f0f0;
    }
    .container {
      background: white;
      padding: 30px;
      border-radius: 10px;
      max-width: 400px;
      margin: 0 auto;
      box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    }
    button {
      padding: 15px 30px;
      font-size: 18px;
      border: none;
      border-radius: 5px;
      cursor: pointer;
      transition: all 0.3s;
      background-color: #007bff;
      color: white;
    }
    button:hover {
      transform: scale(1.05);
    }
    .info {
      margin-top: 20px;
      color: #666;
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>Question 2: Dynamic Button</h2>
    <div id="button-container"></div>
    <div class="info">Click the button to change its color
to green</div>
  </div>

  <script>

    let button = document.createElement("button");
    button.textContent = "Click to Turn Green";
```

```
        button.addEventListener("click", function() {
            this.style.backgroundColor = "#28a745";
            this.textContent = "✓ Turned Green!";
            console.log("Button clicked and color changed to
green");
        });

        document.getElementById("button-
container").appendChild(button);
    </script>
</body>
</html>
```

Solution 3: Advanced Version (multiple colors)

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Question 2 - Advanced Demo</title>
  <style>
    body {
      font-family: Arial;
      padding: 50px;
      background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%);
      min-height: 100vh;
    }
    .card {
      background: white;
      max-width: 500px;
      margin: 0 auto;
      padding: 30px;
      border-radius: 15px;
      box-shadow: 0 10px 30px rgba(0,0,0,0.2);
      text-align: center;
    }
    .dynamic-btn {
      padding: 15px 30px;
      font-size: 18px;
      border: none;
      border-radius: 8px;
      cursor: pointer;
      transition: all 0.3s;
      background-color: #dc3545;
      color: white;
      margin: 10px;
    }
    .dynamic-btn:hover {
      transform: translateY(-2px);
      box-shadow: 0 5px 15px rgba(0,0,0,0.2);
    }
    .log {
      margin-top: 20px;
      padding: 10px;
      background: #f8f9fa;
      border-radius: 5px;
      font-family: monospace;
    }
  </style>
</head>
<body>
  <div class="card">
    <h1>Question 2 Solution</h1>
    <p>Button created entirely with JavaScript</p>
    <div id="button-area"></div>
```

```

    <div class="log" id="log"></div>
</div>

<script>

    let dynamicBtn = document.createElement("button");
    dynamicBtn.className = "dynamic-btn";
    dynamicBtn.textContent = "Click Me! 🖱️ ";

    dynamicBtn.addEventListener("click", function(event) {

        this.style.backgroundColor = "#28a745";
        this.textContent = "✓ Green! ✓";

        let log = document.getElementById("log");
        log.innerHTML = `👉 ✓ Button clicked at ${new
Date().toLocaleTimeString()}<br>Color changed to green`;

    });

    dynamicBtn.addEventListener("mouseenter", function() {
        69) if(this.style.backgroundColor !== "rgb(40, 167,
        this.style.backgroundColor = "#c82333";
    });

    dynamicBtn.addEventListener("mouseleave", function() {
        69) if(this.style.backgroundColor !== "rgb(40, 167,
        this.style.backgroundColor = "#dc3545";
    });

    document.getElementById("button-
area").appendChild(dynamicBtn);

    console.log("Button created successfully!");
</script>
</body>
</html>

```

QUESTION 3: Input Validation with Live Heading Update

Requirements:

- Input element with placeholder "enter your name"
- H2 heading in HTML
- Only allow letters (a-z, A-Z) and spaces
- Dynamically show input in heading
- Invalid characters should NOT appear in heading

Solution 1: Basic Version

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Question 3 - Name Input Validator</title>
  <style>
    body {
      font-family: Arial;
      padding: 50px;
      max-width: 500px;
      margin: 0 auto;
    }
    input {
      width: 100%;
      padding: 10px;
      font-size: 16px;
      border: 2px solid #ddd;
      border-radius: 5px;
      margin: 10px 0;
    }
    h2 {
      background: #f0f0f0;
      padding: 15px;
      border-radius: 5px;
      min-height: 50px;
    }
    .error {
      color: red;
      font-size: 12px;
      margin-top: 5px;
    }
  </style>
</head>
<body>
  <h1>Name Input Demo</h1>

  <h2 id="displayName">Your name will appear here</h2>

  <input type="text" id="nameInput" placeholder="enter your name">

  <div class="error" id="errorMsg"></div>

  <script>
    let input = document.getElementById("nameInput");
    let heading = document.getElementById("displayName");
    let errorMsg = document.getElementById("errorMsg");
```

```

function validateAndUpdate() {
    let rawValue = input.value;
    let filteredValue = "";
    let invalidChars = [];

    for(let i = 0; i < rawValue.length; i++) {
        let char = rawValue[i];

        if((char >= 'a' && char <= 'z') ||
            (char >= 'A' && char <= 'Z') ||
            char === ' ') {
            filteredValue += char;
        } else {
            invalidChars.push(char);
        }
    }

    if(filteredValue !== rawValue) {
        input.value = filteredValue;
        errorMsg.textContent = `Invalid characters
removed: ${invalidChars.join(', ')}`;
        setTimeout(() => {
            errorMsg.textContent = "";
        }, 2000);
    } else {
        errorMsg.textContent = "";
    }

    if(filteredValue.trim() === "") {
        heading.textContent = "Your name will appear
here";
    } else {
        heading.textContent = filteredValue;
    }
}

input.addEventListener("input", validateAndUpdate);
</script>
</body>
</html>

```

Solution 2: Advanced Version with Real-time Validation

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Question 3 - Advanced Name Validator</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana,
sans-serif;
      background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%);
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 20px;
    }

    .container {
      background: white;
      border-radius: 20px;
      padding: 40px;
      max-width: 600px;
      width: 100%;
      box-shadow: 0 20px 60px rgba(0,0,0,0.3);
    }

    h1 {
      color: #333;
      margin-bottom: 20px;
      text-align: center;
    }

    /* H2 heading styling */
    .name-display {
      background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%);
      color: white;
      padding: 20px;
      border-radius: 15px;
      text-align: center;
      margin-bottom: 30px;
      min-height: 80px;
      display: flex;
      align-items: center;
```

```
        justify-content: center;
        word-break: break-word;
        box-shadow: 0 5px 15px rgba(0,0,0,0.1);
    }

    .input-group {
        margin-bottom: 20px;
    }

    label {
        display: block;
        margin-bottom: 8px;
        color: #555;
        font-weight: bold;
    }

    input {
        width: 100%;
        padding: 12px 15px;
        font-size: 16px;
        border: 2px solid #e0e0e0;
        border-radius: 10px;
        transition: all 0.3s;
    }

    input:focus {
        outline: none;
        border-color: #667eea;
        box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
    }

    .validation-status {
        margin-top: 10px;
        padding: 10px;
        border-radius: 8px;
        font-size: 14px;
    }

    .status-valid {
        background: #d4edda;
        color: #155724;
        border: 1px solid #c3e6cb;
    }

    .status-invalid {
        background: #f8d7da;
        color: #721c24;
        border: 1px solid #f5c6cb;
    }

    .char-counter {
```

```

        text-align: right;
        font-size: 12px;
        color: #666;
        margin-top: 5px;
    }

    .allowed-chars {
        background: #f8f9fa;
        padding: 10px;
        border-radius: 8px;
        font-size: 12px;
        color: #666;
        margin-top: 20px;
        text-align: center;
    }

    .badge {
        display: inline-block;
        background: #28a745;
        color: white;
        padding: 2px 8px;
        border-radius: 12px;
        font-size: 11px;
        margin: 0 2px;
    }
</style>
</head>
<body>
    <div class="container">
        <h1>🌟 Name Input Validator 🌟 </h1>

        <div class="name-display">
            <h2 id="liveHeading">Your name will appear
here</h2>
        </div>

        <div class="input-group">
            <label>Enter your name:</label>
            <input type="text" id="nameField"
placeholder="enter your name" autocomplete="off">
            <div class="char-counter">
                <span id="charCount">0</span> characters
            </div>
        </div>

        <div id="statusMessage" class="validation-status">
</div>

        <div class="allowed-chars">
            🟢 Allowed characters:

```

```

    <span class="badge">A-Z</span>
    <span class="badge">a-z</span>
    <span class="badge">Space</span>
    <br>
    ✖ Numbers, symbols, and special characters are
    automatically filtered out
  </div>
</div>

<script>

  let inputField = document.getElementById("nameField");
  let heading = document.getElementById("liveHeading");
  let statusDiv =
document.getElementById("statusMessage");
  let charCountSpan =
document.getElementById("charCount");

  function isAllowedChar(char) {

    return (char >= 'a' && char <= 'z') ||
      (char >= 'A' && char <= 'Z') ||
      char === ' ';
  }

  function filterInput(inputString) {
    let result = "";
    let invalidChars = [];

    for(let i = 0; i < inputString.length; i++) {
      let char = inputString[i];
      if(isAllowedChar(char)) {
        result += char;
      } else {
        invalidChars.push(char);
      }
    }

    return { filtered: result, invalid: invalidChars
  };
}

function handleNameInput() {
  let rawValue = inputField.value;
  let { filtered, invalid } = filterInput(rawValue);

  if(filtered !== rawValue) {

```

```

        inputField.value = filtered;
    }

    if(filtered.trim() === "") {
        heading.textContent = "Your name will appear
here";
        heading.style.opacity = "0.7";
    } else {
        heading.textContent = filtered;
        heading.style.opacity = "1";
    }

    charCountSpan.textContent = filtered.length;

    if(invalid.length > 0) {
        statusDiv.className = "validation-status
status-invalid";
        statusDiv.innerHTML = `
            ⚠ Invalid character${invalid.length > 1 ?
's' : ''} removed:
            <strong>${invalid.join(', ')}</strong>
            <br>Only letters and spaces are allowed!
        `;

        setTimeout(() => {
            if(statusDiv.className === "validation-
status status-invalid") {
                statusDiv.className = "validation-
status status-valid";
                statusDiv.innerHTML = "✓ Valid name
format";
            }
        }, 2000);
    } else if(filtered.length > 0) {
        statusDiv.className = "validation-status
status-valid";
        statusDiv.innerHTML = "✓ Valid name format";
    } else {
        statusDiv.className = "validation-status";
        statusDiv.innerHTML = "Start typing your
name...";
    }

    console.log(`Input: "${rawValue}" → Filtered:
"${filtered}"`);
}

```

```
        inputField.addEventListener("input", handleNameInput);

        inputField.addEventListener("paste", function(e) {

            setTimeout(handleNameInput, 10);

        });

        handleNameInput();

        console.log("Question 3 solution loaded successfully!");
        console.log("Only letters (A-Z, a-z) and spaces are allowed");
    </script>
</body>
</html>
```

Solution 3: Complete Working Demo with All Features

html

```
<!DOCTYPE html>
<html>
<head>
  <title>Question 3 - Complete Solution</title>
  <meta charset="UTF-8">
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Courier New', monospace;
      background: #0a0e27;
      min-height: 100vh;
      display: flex;
      justify-content: center;
      align-items: center;
      padding: 20px;
    }

    .card {
      background: white;
      border-radius: 20px;
      padding: 40px;
      max-width: 650px;
      width: 100%;
      box-shadow: 0 25px 50px rgba(0,0,0,0.3);
    }

    h1 {
      color: #0a0e27;
      margin-bottom: 10px;
      font-size: 28px;
    }

    .subtitle {
      color: #666;
      margin-bottom: 30px;
      border-bottom: 2px solid #eee;
      padding-bottom: 10px;
    }

    /* Live Heading Display */
    .heading-container {
      background: linear-gradient(135deg, #0a0e27 0%,
#1a1f3a 100%);
      border-radius: 15px;
      padding: 25px;
```

```
        margin-bottom: 30px;
        text-align: center;
        box-shadow: 0 10px 20px rgba(0,0,0,0.1);
    }

    #liveNameHeading {
        color: white;
        font-size: 32px;
        word-break: break-word;
        transition: all 0.3s;
    }

    .placeholder-heading {
        opacity: 0.6;
        font-style: italic;
    }

    /* Input Section */
    .input-section {
        margin-bottom: 25px;
    }

    label {
        display: block;
        margin-bottom: 8px;
        font-weight: bold;
        color: #333;
    }

    #nameInput {
        width: 100%;
        padding: 15px;
        font-size: 16px;
        border: 2px solid #e0e0e0;
        border-radius: 10px;
        transition: all 0.3s;
        font-family: 'Courier New', monospace;
    }

    #nameInput:focus {
        outline: none;
        border-color: #0a0e27;
        box-shadow: 0 0 0 3px rgba(10, 14, 39, 0.1);
    }

    /* Stats */
    .stats {
        display: flex;
        justify-content: space-between;
        margin-bottom: 20px;
        padding: 15px;
```

```
        background: #f8f9fa;
        border-radius: 10px;
    }

    .stat {
        text-align: center;
    }

    .stat-value {
        font-size: 24px;
        font-weight: bold;
        color: #0a0e27;
    }

    .stat-label {
        font-size: 12px;
        color: #666;
        margin-top: 5px;
    }

    /* Messages */
    .message {
        padding: 12px;
        border-radius: 8px;
        margin-bottom: 20px;
        font-size: 14px;
        transition: all 0.3s;
    }

    .message-success {
        background: #d4edda;
        color: #155724;
        border-left: 4px solid #28a745;
    }

    .message-error {
        background: #f8d7da;
        color: #721c24;
        border-left: 4px solid #dc3545;
    }

    .message-info {
        background: #d1ecf1;
        color: #0c5460;
        border-left: 4px solid #17a2b8;
    }

    /* Badges */
    .badge-container {
        display: flex;
        gap: 10px;
```

```

        flex-wrap: wrap;
        margin-top: 20px;
        padding-top: 20px;
        border-top: 1px solid #eee;
    }

    .badge {
        background: #f0f0f0;
        padding: 5px 12px;
        border-radius: 20px;
        font-size: 12px;
        color: #555;
    }

    .badge-allowed {
        background: #d4edda;
        color: #155724;
    }

    /* Animation */
    @keyframes pulse {
        0% { transform: scale(1); }
        50% { transform: scale(1.02); }
        100% { transform: scale(1); }
    }

    .update-animation {
        animation: pulse 0.3s ease;
    }
</style>
</head>
<body>
    <div class="card">
        <h1>  Question 3: Name Validator</h1>
        <div class="subtitle">Only letters (A-Z, a-z) and
spaces allowed</div>

        <div class="heading-container">
            <h2 id="liveNameHeading">Enter your name
above</h2>
        </div>

        <div class="input-section">
            <label>  Enter your name:</label>
            <input type="text"
                id="nameInput"
                placeholder="enter your name"
                autocomplete="off"
                spellcheck="false">

```

```

    </div>

    <div class="stats">
      <div class="stat">
        <div class="stat-value" id="charCount">0</div>
        <div class="stat-label">Characters</div>
      </div>
      <div class="stat">
        <div class="stat-value"
id="letterCount">0</div>
        <div class="stat-label">Letters</div>
      </div>
      <div class="stat">
        <div class="stat-value"
id="spaceCount">0</div>
        <div class="stat-label">Spaces</div>
      </div>
    </div>

    <div id="statusMessage" class="message message-info">
      💡 Start typing your name...
    </div>

    <div class="badge-container">
      <span class="badge badge-allowed">✅ A-Z</span>
      <span class="badge badge-allowed">✅ a-z</span>
      <span class="badge badge-allowed">✅ Space</span>
      <span class="badge">❌ Numbers (0-9)</span>
      <span class="badge">❌ Symbols (!@#%$)</span>
      <span class="badge">❌ Special chars</span>
    </div>
  </div>

  <script>

    let inputField = document.getElementById("nameInput");
    let heading =
document.getElementById("liveNameHeading");
    let statusDiv =
document.getElementById("statusMessage");
    let charCountSpan =
document.getElementById("charCount");
    let letterCountSpan =
document.getElementById("letterCount");
    let spaceCountSpan =
document.getElementById("spaceCount");

```

```
const ALLOWED_CHARS = /[A-Za-z ]/;

function isValidCharacter(char) {
  return ALLOWED_CHARS.test(char);
}

function filterInvalidChars(inputStr) {
  let result = '';
  let invalidChars = [];

  for(let i = 0; i < inputStr.length; i++) {
    let char = inputStr[i];
    if(isValidCharacter(char)) {
      result += char;
    } else {
      invalidChars.push(char);
    }
  }

  return {
    filtered: result,
    invalid: invalidChars,
    hasInvalid: invalidChars.length > 0
  };
}

function countCharacters(str) {
  let letters = 0;
  let spaces = 0;

  for(let i = 0; i < str.length; i++) {
    let char = str[i];
    if(char === ' ') {
      spaces++;
    } else if(char >= 'A' && char <= 'Z' || char
>= 'a' && char <= 'z') {
      letters++;
    }
  }

  return { letters, spaces, total: str.length };
}

function animateHeading() {
  heading.classList.add('update-animation');
  setTimeout(() => {
    heading.classList.remove('update-animation');
  }, 1000);
}
```

```

    }, 300);
  }

  function updateHeadingAndInput() {

    let rawValue = inputField.value;

    let { filtered, invalid, hasInvalid } =
    filterInvalidChars(rawValue);

    if(filtered !== rawValue) {
      inputField.value = filtered;

      if(hasInvalid) {
        let uniqueInvalid = [...new Set(invalid)];
        statusDiv.className = "message message-
error";

        statusDiv.innerHTML = `❌ Removed invalid
character${uniqueInvalid.length > 1 ? 's' : ''}:
<strong>${uniqueInvalid.join(', ')}</strong><br>Only letters
and spaces are allowed!`;

        setTimeout(() => {
          if(statusDiv.className === "message
message-error") {
            if(filtered.length > 0) {
              statusDiv.className = "message
message-success";

              statusDiv.innerHTML = "✅
Valid name! All characters are allowed.";
            } else {
              statusDiv.className = "message
message-info";

              statusDiv.innerHTML = "💡
Enter your name (letters and spaces only)";
            }
          }
        }, 2000);
      }
    } else {

      if(filtered.length > 0) {
        statusDiv.className = "message message-
success";

        statusDiv.innerHTML = "✅ Valid name! All
characters are allowed.";
      }
    }
  }
}

```

```

        } else {
            statusDiv.className = "message message-
info";

            statusDiv.innerHTML = "💡 Enter your name
(letters and spaces only)";
        }
    }

    if(filtered.trim() === "") {
        heading.textContent = "✨ Your name will
appear here ✨";
        heading.classList.add("placeholder-heading");
    } else {
        heading.textContent = filtered;
        heading.classList.remove("placeholder-
heading");
        animateHeading();
    }

    let counts = countCharacters(filtered);
    charCountSpan.textContent = counts.total;
    letterCountSpan.textContent = counts.letters;
    spaceCountSpan.textContent = counts.spaces;

    console.log(`Input: "${rawValue}" → Filtered:
"${filtered}" | Invalid: ${invalid.join(',') || 'none'}`);
}

function handlePaste(e) {

    setTimeout(updateHeadingAndInput, 10);
}

function handleKeyDown(e) {

    const controlKeys = [
        'Backspace', 'Delete', 'ArrowLeft',
'ArrowRight',
        'ArrowUp', 'ArrowDown', 'Tab', 'Home', 'End',
        'Enter', 'Ctrl', 'Alt', 'Shift', 'Meta'
    ];

    if(controlKeys.includes(e.key)) {
        return;
    }

```

```

        if(!isValidCharacter(e.key) && e.key !== ' ') {
            e.preventDefault();
            statusDiv.className = "message message-error";
            statusDiv.innerHTML = `⚠️ "${e.key}" is not
allowed! Only letters and spaces.`;

            setTimeout(() => {
                if(statusDiv.className === "message
message-error" && inputField.value.length > 0) {
                    statusDiv.className = "message
message-success";
                    statusDiv.innerHTML = "✅ Valid
name!";
                } else if(statusDiv.className === "message
message-error") {
                    statusDiv.className = "message
message-info";
                    statusDiv.innerHTML = "💡 Enter your
name (letters and spaces only)";
                }
            }, 1500);
        }
    }

    inputField.addEventListener("input",
updateHeadingAndInput);
    inputField.addEventListener("paste", handlePaste);
    inputField.addEventListener("keydown", handleKeyDown);

    inputField.addEventListener("change",
updateHeadingAndInput);

    updateHeadingAndInput();

    console.log("🌟 Question 3 Solution Ready! 🌟");
    console.log("Features:");
    console.log("✓ Only A-Z, a-z, and spaces allowed");
    console.log("✓ Invalid characters automatically
filtered");
    console.log("✓ Heading updates in real-time");
    console.log("✓ Shows statistics (characters, letters,
spaces)");
    console.log("✓ Visual feedback with animations");

    console.log("\n🖱️ Try typing these examples:");

```

```

        console.log("    ✓ 'John Doe' - Valid");
        console.log("    ✓ 'Marie Antoinette' - Valid");
        console.log("    ✗ 'John123' - Numbers will be
removed");
        console.log("    ✗ 'Hello@World' - Symbol will be
removed");
    </script>
</body>
</html>

```

Key Points Summary

Question 2:

javascript

```

let button = document.createElement("button");
button.addEventListener("click", function() {
    this.style.backgroundColor = "green";
});

```

Question 3:

javascript

```

input.addEventListener("input", function() {
    let filtered = this.value.replace(/[^A-Za-z ]/g, '');
    heading.textContent = filtered || "Enter your name";
    this.value = filtered;
});

```

Testing Both Questions:

1. Save any of the solutions as `.html` files
2. Open in browser
3. For Q2: Click button → turns green
4. For Q3: Type letters/space → heading updates; Type numbers/symbols → filtered out

Both solutions fully meet the requirements! 🎯

